

# Attributes reflection

Document #: P3385R5 [[Latest](#)] [[Status](#)]  
Date: 2025-05-18  
Project: Programming Language C++  
Audience: EWG, LEWG  
Reply-to: Aurelien Cassagnes  
<[acassagnes@bloomberg.net](mailto:acassagnes@bloomberg.net)>

## Contents

- [1 Revision history](#)
- [2 Introduction](#)
  - [2.1 Earlier work](#)
  - [2.2 Scope](#)
  - [2.3 Self consistency and optionality rule](#)
- [3 Proposed Features](#)
  - [3.1 info](#)
  - [3.2 Reflection operator](#)
  - [3.3 Splicers](#)
  - [3.4 Metafunctions](#)
  - [3.5 Queries](#)
- [4 Proposed wording](#)
  - [4.1 Language](#)
  - [4.2 Library](#)
  - [4.3 Feature-test macro](#)
- [5 Feedback](#)
  - [5.1 Poll](#)
  - [5.2 Implementation](#)
- [6 Conclusion](#)
- [7 References](#)

## § 1 Revision history

Since [\[P3385R4\]](#)

- Remove in-place splicer syntax
- Add appertain metafunction
- Augment implementation feedback

## § 2 Introduction

Attributes are used to a great extent, and there is new attributes being added to the language somewhat regularly.

As reflection makes its way into our standard, we are missing a way for generic code to look into the attributes appertaining to an entity. That is what this proposal aims to tackle by introducing the building blocks.

We expect a number of applications for attribute introspection to happen in the context of code injection [\[P2237R0\]](#), where, for example, one may want to skip over `[[deprecated]]` members. The following example demonstrates skipping over deprecated members:

```
struct User {
    [[deprecated]] std::string name;
    [[deprecated]] std::string country;
    std::string uuidv5;
    std::string countryIsoCode;
};

constexpr std::meta::info filterOutDeprecated() {
    std::vector<info> liveMembers;
    constexpr auto attributes = attributes_of(^^[[deprecated]]);
    auto keepLive = [&] (info r) {
        if (!std::ranges::any_of(
            attributes_of(^^r),
            [&] (auto meta) { meta == attributes[0]; }
        )) {
            liveMembers.push_back(r);
        }
    };

    // Migrated user will no longer support deprecated fields
    struct MigratedUser;
    constexpr { define_aggregate(^^MigratedUser, selectNonDeprecated()); }

    template for (constexpr auto member : members_of(^^User)) {
        keepLive(member);
    }
    return ^^MigratedUser;
}
```

## § 2.1 Earlier work

While collecting feedback on this draft, we were redirected to [P1887R1], a pre existing proposal. In this paper, the author discusses two topics: ‘user defined attributes’ (also in [P2565R0]) and reflection over said attributes. We believe these two topics need not be conflated; both have intrinsic values on their own. We aim to focus this discussion entirely on **standard** attributes reflection. Furthermore the earlier paper has not seen work following the progression of [P2996R11], and so we feel this proposal is in a good place to fill that gap.

## § 2.2 Scope

### § 2.2.1 Standard vs arbitrary attributes

Attributes are colloquially split into standard and non-standard. The current wording in the standard states that attributes which are not recognized are ignored, without being clear on what *ignoring* means here. When we speak about reflection and attributes, what matter is whether they are kept alive until semantic analysis. While this is true for standard attributes, regardless of whether recommendations from the standard are applied (eg. Warning on ignoring a nodiscard marked entity), it is less clear whether that holds as well for non-standard ones. The proposal does not ask for implementation to change their current scheme, how “non standard attributes”, “reflection” and “ignorability” interact together will be discussed further in following [paragraph](#).

To summarize, this proposal (via wording) wishes to be prescriptive when it comes to standard attributes (9.13 [\[dcl.attr\]](#)) and permissive when it comes to non-standard.

### § 2.2.2 Argument clause

Revisions of this proposal up to [P3385R2] were willfully ignoring *attribute-argument-clause*, prescriptively so when it comes to comparison (ie. `^[nodiscard("foo")] == ^[nodiscard("bar")]`). Feedback post Wroclaw was unanimous on the need to support argument clause. Feedback from implementers on this feature has been that while it brings no concern for attributes like nodiscard, it was unrealistic in the case of assume that accepts an arbitrary expression as argument.

Note that this is at this point a somewhat artificial concern as there is no meaningful way to reflect on a null statement, which is what assume appertains to. The only way to get a reflection of assume would be to explicitly construct one via `constexpr auto r = ^[assume(expr)];`<sup>1</sup>. For any other entity, there is no call to `attributes_of` that could return a reflection of assume attribute.

Let us recap here what are the attributes 9.13 [\[dcl.attr\]](#) found in the standard and their argument clause

| Attribute          | Argument-clause        |
|--------------------|------------------------|
| assume             | conditional-expression |
| carries_dependency | N/A                    |
| deprecated         | unevaluated-string     |
| fallthrough        | N/A                    |
| indeterminate      | N/A                    |
| likely             | N/A                    |
| maybe_unused       | N/A                    |
| nodiscard          | unevaluated-string     |
| noreturn           | N/A                    |
| no_unique_address  | N/A                    |
| unlikely           | N/A                    |

Besides the assume case, the arguments are unproblematic.

On the other hand if we look at implementation specific attributes, we'll see that arguments can take arbitrary shape; without listing (all) clang recognized attributes, we'll share one example here

```
[[clang::availability(macos,introduced=10.4,deprecated=10.6,obsoleted=10.7)]]
void f();
```

There is no standard way to express the shape of the arguments (*list of tag = value where tags are from a predetermined set ?*) to be supported here, short of treating those arguments clause as unevaluated soup of arbitrary tokens... Arguably when the generative side of reflection does pick up momentum, this will be revisited.

In the end, the current proposal aims to cover the most realistic use cases today by rendering ill formed the reflection of attributes with *conditional-expression* argument clause, leaving it open for future standard to lift that restriction.

## § 2.3 Self consistency and optionality rule

There is a long standing and confusing discussion around the ignorability of attributes. We'll refer the reader to [P2552R3] for an at-length discussion of this problem, and especially with regard to what 'ignorability' really means. Another interesting conversation takes place in [P3254R0], around the [[no\_unique\_address]] case, which serves again to illustrate that the tension around so called ignorability should not be considered a novel feature of this proposal.

We claim that whether an implementation decides to semantically ignore a standard attribute (by not applying the *recommended practices*) does not matter in the context of this proposal. What does matter is that attributes and reflection interact in a self-consistent fashion. To that aim we think the following guidelines provide a decent sandbox

1. Reflecting on an ignored attribute shall be undistinguishable from reflecting on `[[ ]]`.
2. Splicing attributes on a declaration shall be undistinguishable from manually appertaining those attributes to this declaration.

#### About the first rule

For *standard attribute*, the first rule does not come into the picture as they are not syntactically ignorable.

For *implementation specific attributes* <sup>2</sup>, if a particular implementation wishes to ignore this attribute then the first rule says the implementation should treat this as an empty attribute list.

#### About the second rule

For *standard attribute*, this rule simply enforce appertainance rule as they are usually prescribed in the standard.

For *implementation specific attributes*, this rule means that if an implementation choses to ignore a particular attribute, splicing a reflection of that attribute will trigger the same ignorability trap.

## § 3 Proposed Features

We put ourselves in the context of [\[P2996R11\]](#) for this proposal to be more illustrative in terms of what is being proposed.

### § 3.1 info

We propose that attributes be a supported *reflectable* property of the expression that is reflected upon. That means value of type `std::meta::info` should be able to represent an attribute in addition to the currently supported set.

### § 3.2 Reflection operator

The current proposition for reflection operator grammar does not cover attributes, i.e., the expression `^^[[deprecated]]` is ill-formed. Our proposal advocates to support expression as following:

```
constexpr auto keepAttribute = ^^[[nodiscard]];
```

The resulting value is a reflection value embedding salient property of the attribute which are the token and the attribute argument clause if any. If the *attribute* is not a standard attribute, as per our earlier discussion, an implementation is free to either compute that reflection or to treat this as a reflection over a vanishing attribute.

### § 3.3 Splicers

There is **no** support for splicing attributes in place. Up to and including [P3385R4] we were offering that a splice syntax be supported to splice attributes at declaration location

```
[[ [: r :] ]]
```

A simple example of splicer is as follows where we create an augmented `enum` introducing a begin and end enumerator while preserving the original attributes:

```
enum class [[nodiscard]] ErrorCode {
    warn,
    fatal,
};

enum class [[ [: ^^ErrorCode :] ]] ClosedErrorCode { // identical to
    `enum class [[nodiscard]] ClosedErrorCode {`
    begin,
    // ...
    end,
};
```

Since we feel the lookup inside `[[ ]]` needs to be designed more carefully than what we were offering in our proposal, we are removing support for *in-place* splicing attributes in place, in favor of an `appertain()` metafunction. We will in the future review carefully how that feature should be designed if at all.

## § 3.4 Metafunctions

We propose to add couple of metafunctions to what has already been discussed in [P2996R11]. In addition, we will extend support to attributes in the other metafunctions, when it makes sense.

### § 3.4.1 attributes\_of

```
namespace std::meta {
    consteval auto attributes_of(info entity) -> vector<info>;
}
```

This would return a vector of reflections representing individual attributes that were appertaining to reflected upon entity.

```
enum class [[nodiscard]] ErrorCode {
    Disconnected,
    ConfigurationIncorrect,
    OutdatedCredentials,
};

static_assert(attributes_of(^^ErrorCode)[0] == ^^[[nodiscard]]);
```

### § 3.4.2 is\_attribute

```
namespace std::meta {
    consteval auto is_attribute(info entity) -> bool;
}
```

This would return true if the entity reflection represents a [ [ attribute ] ] such as described in [dcl.attr], otherwise, it would return false. For vendor specific attributes, behavior is left to implementation.

```
static_assert(is_attribute(^^[[nodiscard]]));
static_assert(is_attribute(^^[[clang::disable_tail_calls]])); //
    Implementation defined
```

### § 3.4.3 appertain

```
namespace std::meta {
    template <reflection_range R = initializer_list<info>>
    consteval auto appertain(info entity, R&& attributes) -> info;
}
```

This would appertain to entity each attribute reflection in attributes.

```
struct Foo;

consteval {
    define_aggregate(^^Foo, {});
}
constexpr auto r = appertain(^^Foo, { ^^[[maybe_unused]] });
static_assert(attributes_of(^^r)[0] == ^^[[maybe_unused]]);
```

### § 3.4.4 identifier\_of, display\_string\_of

Given a reflection *r* designating a standard attribute, *identifier\_of(r)* (resp. *u8identifier\_of(r)*) should return a *string\_view* (resp. *u8string\_view*) corresponding to the attribute-token. We do not think the leading [ [ and closing ] ] are meaningful, besides, they contribute visual noise.

A sample follows

```
[[nodiscard]] int func();
constexpr auto nodiscard = attributes_of(^^func);
    static_assert(identifier_of(nodiscard[0]) ==
    identifier_of(^^[[nodiscard]]));
```

```
static_assert(identifier_of(nodiscard[0]) == "nodiscard"); // != "[nodiscard]"
```

Given a reflection `r` that designates an individual attribute, `display_string_of(r)` (resp. `u8display_string_of(r)`) returns an unspecified non-empty `string_view` (resp. `u8string_view`). Implementations are encouraged to produce text that is helpful in identifying the reflected attribute for display purpose. In the preceding example we could imagine printing `[[nodiscard]]` instead of `nodiscard` as it might be better fitted for log extraction.

### § 3.4.5 data\_member\_spec, define\_aggregate

As it stands now, `define_aggregate` allows piecewise building of a class via `data_member_spec`. However, to support arbitrary attributes pertaining to those data members, we'll need to augment `data_member_options` to encode attributes we may want to attach to a data member.

The structure would change thusly:

```
namespace std::meta {
    struct data_member_options {
        struct name_type {
            template <typename T> requires constructible_from<u8string, T>
                consteval name_type(T &&);

            template <typename T> requires constructible_from<string, T>
                consteval name_type(T &&);
        };

        optional<name_type> name;
        optional<int> alignment;
        optional<int> bit_width;
        vector<info> attributes;
        bool no_unique_address = false;
        [[deprecated]] bool no_unique_address = false;
    };
}
```

From there building an aggregate piecewise proceeds as usual

```
struct Empty {};  
struct [[nodiscard]] S;  
consteval {  
    define_aggregate(^^S, {  
        data_member_spec(^^int, {.name = "i"}),  
        data_member_spec(^^Empty, {.name = "e"},  
                           .attributes =  
                               {^^[[no_unique_address]]})  
    });  
};
```



```

}

// Equivalent to
// struct [[nodiscard]] S {
//     int i;
//     [[no_unique_address]] struct Empty { } e;
// };

```

Note here that [\[P2996R11\]](#) includes `no_unique_address` and `alignment` as part of `data_member_options` API. We think that this approach scale awkwardly in that every new attributes introduced into the standard will lead to discussions on whether or not they should be included in `data_member_options`. Attaching attributes through the above proposed approach is more in line with the philosophy of leveraging `info` as the opaque vehicle to carry every and all reflections.

We will not pursue this change here in this proposal but in a follow-up paper [P3678R0?](#), as we wish to keep the scope of change here rather small.

### § 3.4.6 Other metafunctions

For any reflection where `is_attribute` returns `true`, other metafunctions not listed above are not considered constant expressions

## § 3.5 Queries

We do not think it is necessary to introduce any additional query or queries at this point. We would especially not recommend introducing a dedicated query per attribute (e.g., `is_deprecated`, `is_nouniqueaddress`, etc.). Having said that, we feel those should be achievable via concepts, something akin to:

```

constexpr auto deprecated = attributes_of(^^[[deprecated]])[0];

template<class T>
concept IsDeprecated = std::ranges::any_of(
    attributes_of(^T),
    [[deprecated]] (auto meta) { meta == deprecated; }
);

```

## § 4 Proposed wording

### § 4.1 Language

#### § 6.8.2 [\[basic.fundamental\]](#) Fundamental types

Augment the description of `std::meta::info` found in new paragraph 17.1 to add standard attribute as a valid representation to the current enumerated list

- 17-1 A value of type `std::meta::info` is called a *reflection*. There exists a unique *null reflection*; every other reflection is a representation of
- ...
  - an attribute
  - a data member description (11.4.1 [class.mem.general]).

Update 17.2 Recommended practices to remove attributes from the list

- 17-2 *Recommended practice*: Implementations are discouraged from representing any constructs described by this document that are not explicitly enumerated in the list above (e.g., partial template specializations, ~~attributes~~, placeholder types, statements).

### § 7.6.2.10\* [expr.reflect] The reflection operator

Edit ^^ operator grammar for attribute reflection

```
reflect-expression:
  ^^ ::
  ^^ unqualified-id
  ^^ qualified-id
  ^^ type-id
  ^^ pack-index-expression
  ^^ [[ attribute ]]
```

Add the following paragraph at the bottom of 7.6.2.10 to describe: 1) the reflection of attributes, 2) what implementation shall yield when reflecting unsupported attributes, 3) forbidding the reflection of `[[assume(conditional-expression)]]`

- (11.1) A *reflect-expression* of the form `^^[[ attribute ]]` computes a reflection of the attribute 9.13 [dcl.attr]. For an *attribute-token* not described in this document, an implementation ignoring such *attribute-token* shall return a computed value equal to `^^[ ]`.
- (11.2) For an attribute *r* described in this document whose *attribute-argument-clause* is present and is a *conditional-expression*, computing the reflection of *r* is ill-formed.

### § 7.6.10 [expr.eq] Equality Operators

Update new paragraph between 7.6.10 [expr.eq]/5 and /6 to add a clause for comparing reflection of attributes and renumber accordingly

If both operands are of type `std::meta::info`, comparison is defined as follows:

- (5+.5) - Otherwise, if one operand represents a direct base class relationship, then they compare equal if and only if the other operand represents the same direct base class relationship.

- (5+.) - Otherwise if one operand represents an attribute, then they compare equal if and only if the other operand represents an attribute, both those attributes have the same *attribute-token*, and if present, both *attribute-argument-clause* are equal.  
[ *Example:*

```
static_assert(^^[nodiscard] == ^^[nodiscard]);  
    static_assert(^^[nodiscard("keep me")] ==  
    ^^[nodiscard("keep me")]);  
    static_assert(^^[nodiscard("keep me")] !=  
    ^^[nodiscard("keep me too")]);  
static_assert(^^[nodiscard("keep me")] != ^^[nodiscard]);  
static_assert(^^[nodiscard] != ^^[deprecated]);
```

— *end example* ]

- (5+.6) - Otherwise, both operands  $0_1$  and  $0_2$  represent data member descriptions. The operands compare equal if and only if the data member descriptions represented by  $0_1$  and  $0_2$  compare equal (11.4.1 [class.mem.general]).

## § 4.2 Library

### § 4.2.1 [meta.reflection.synop] Header **<meta>** synopsis

Add to the [meta.reflection.queries] section from the synopsis, the metafunction `is_attribute`. Add to the new [meta.reflection.attribute] the metafunctions `attributes_of` and `appertain`

```
...  
// [meta.reflection.queries], reflection queries  
...  
constexpr bool is_attribute(info r);  
...  
// [meta.reflection.attribute], attribute reflection  
...  
constexpr vector<info> attributes_of(info r);  
template <reflection_range R = initializer_list<info>>  
constexpr info appertain(info r, R&& v);
```

### § 4.2.2 [meta.reflection.queries] Reflection queries

Describe the behavior for `is_attribute` metafunction, returning true for reflection of attribute and false otherwise.

```
* constexpr bool is_attribute(info r);
```

*Returns:* true if `r` represents an attribute. Otherwise, false.

[ Example:

```
static_assert(is_attribute(^^[[nodiscard]]));
```

— end example ]

### § 4.2.3 [meta.reflection.attribute] Attribute reflection

Describe the behavior for `attributes_of` metafunction, returning a sequence of reflection of each attributes appertaining to `r`. The returned sequence is sorted in implementation-specific order

```
* constexpr vector<info> attributes_of(info r);
```

*Returns:* A vector containing reflections of all attributes appertaining to the entity represented by `r`

[ Example:

```
struct [[nodiscard]] Result { Success, Failure };

static_assert(attributes_of(^^Result).size() == 1);
static_assert(attributes_of(^^Result)[0] == ^^[[nodiscard]]);
```

— end example ]

```
* template <reflection_range R = initializer_list<info>>
    constexpr info appertain(info r, R&& v);
```

*Returns:* Return a reflection of the entity `e` originally described by `r`, each attribute found in the sequence `v` is now considered appertaining to `e`

*Constant When:* `dealias(r)` represents a class type, variable, function, or a namespace; and `is_attribute(v_i)` is true for each `v_i` in `v`

[ Example:

```
struct Foo;

constexpr {
    define_aggregate(^^Foo, {});
}
constexpr auto r = appertain(^^Foo, { ^^[[maybe_unused]] });
static_assert(attributes_of(r)[0] == ^^[[maybe_unused]]);
static_assert(attributes_of(^^Foo)[0] == ^^[[maybe_unused]]);
```

— *end example* ]

## § 4.2.4 [meta.reflection.names], Reflection names and locations

Update description of `has_identifier` return value to be `true` for attribute reflection and renumber accordingly

```
constexpr bool has_identifier(info r);
```

1 *Returns:*

...

(1.\*) Otherwise, if `r` represents an attribute, then `true`

(1.10) Otherwise false

Add a paragraph to `identifier_of`, `u8identifier_of` to describe return value of attribute reflection to be the `attribute-token`

```
constexpr string_view identifier_of(info r);  
constexpr string_view u8identifier_of(info r);
```

4 *Returns:*

...

(4.6) Otherwise, if `r` represents an attribute, then the `attribute-token`

## § 4.3 Feature-test macro

The attribute reflection feature is guarded behind macro, augment 15.12 [cpp.predefined]

```
__cpp_impl_reflection 2025XXL  
__cpp_impl_reflection_attributes 2025XXL
```

# § 5 Feedback

## § 5.1 Poll

### § 5.1.1 P3385R1: SG7, Nov 2024, WG21 meetings in Wroclaw

— SG7 encourages more work on reflection of attributes as described in the paper: No objection to unanimous consent

### § 5.1.2 P3385R2: SG7, Dec 2024, Telecon

— SG7 wants to support namespaced attributes: No objection to unanimous consent.

— SG7 wants to support “easy” arguments of attributes: No objection to unanimous consent.

- SG7 wants to support arguments (full expressions) of attributes: Not consensus.
- SG7 considers the paper high-priority and forwards it to LEWG and EWG for C++26: Not consensus.
- SG7 forwards this paper to LEWG and EWG for C++29: Not consensus.

### § 5.1.3 P3385R3: SG7, Feb 2025, Hagenberg

- SG7 wants to support token source type arguments: Not consensus
- SG7 wants to forward to EWG and LEWG as is: Consensus

## § 5.2 Implementation

Most of the features presented here are available on a branch <sup>3</sup> that is being PR to the Bloomberg P2996 Clang implementation. Most of the challenges <sup>4</sup> met so far were around two elements

- Recovering attribute arguments. After an attribute becomes semantic, there is not an unified way to go back to their syntactic forms and from there, extract arguments. Our strategy (hack) was to add a backlink from semantic to syntactic form of attributes, and extend their lifetime so the links remain available. Arguably a better solution is to add an accessor to `Attr` to recover the arguments directly and not have this artificial backlink...
- Splicing dependent expression. We must delay the evaluation of `[[ [: ^^T :] ]]` when `T` is dependent and remember to expand the recovered attributes at instantiation. This was done by creating a custom attribute with argument type expression. At parse time when meeting `[[ [: ^^T :] ]]`, we attached that custom attribute to the AST node under evaluation. Later on when we instantiate the appertained-to entity, that custom attribute is recovered, the dependent expression is evaluated and the attributes extracted.

Note that the second point is rather moot starting from revision 5 where *in-place* splicing of attributes was removed.

## § 6 Conclusion

Originally the idea of introducing a `declattr(Expression)` keyword seemed the most straightforward approach to tackling this problem. However based on feedback, the concern of introspecting on expression attributes was a topic that belongs with the Reflection SG. The current proposal shifted away from the original `declattr` idea to align better with the reflection toolbox. Note also that, as we advocate here for `[[ [: r :] ]]` to be supported, we recover the ease of use that we first envisioned `declattr` to have.

## § 7 References

- [P1887R1] Corentin Jabot. 2020-01-13. Reflection on attributes.  
<https://wg21.link/p1887r1>
- [P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.  
<https://wg21.link/p2237r0>
- [P2552R3] Timur Doumler. 2023-06-14. On the ignorability of standard attributes.  
<https://wg21.link/p2552r3>
- [P2565R0] Bret Brown. 2022-03-16. Supporting User-Defined Attributes.  
<https://wg21.link/p2565r0>
- [P2996R11] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2025-04-16. Reflection for C++26.  
<https://wg21.link/p2996r11>
- [P3254R0] Brian Bi. 2024-05-22. Reserve identifiers preceded by @ for non-ignorable annotation tokens.  
<https://wg21.link/p3254r0>
- [P3385R2] Aurelien Cassagnes, Roman Khoroshikh, Anders Johansson. 2024-12-12. Attributes reflection.  
<https://wg21.link/p3385r2>
- [P3385R4] Aurelien Cassagnes. 2025-03-11. Attributes reflection.  
<https://wg21.link/p3385r4>
- 

1. Oddly enough this is what reflection of a null statement decorated by this attribute would look like, if that was possible.↵
2. Note that we are here going straight to the subset of non standard attributes that matter the most, implementation specific attributes. User defined arbitrary attributes are best addressed via the annotation proposal.↵
3. <https://github.com/zebullax/clang-p2996/tree/p3385>↵
4. Those were challenges for a first time contribution to Clang, they may not be so challenging for other contributors.↵